

پروژه چهارم آزمایشگاه سیستم عامل

مجید سپاهی - 810100163

امیرحسین صباحی - 810101556

بخش اول: تحلیل هسته و قفل های پیشرفته

1. مفهوم مسیر کنترلی (Control Path) را تعریف کنید. تفاوت Syscall، Interrupt و Exception چیست؟

مسیر کنترلی یعنی مسیر اجرای CPU از لحظه وقوع یک رویداد تا پایان رسیدگی به آن رویداد. در Syscall ورود به کرنل عمدی است و خود برنامه کاربر آن را درخواست می‌کند، پس همگام است. در Interrupt ورود به کرنل ناهمگام است و معمولاً سخت‌افزار (مثل تایمر یا دیسک) آن را ایجاد می‌کند. در Exception ورود به کرنل همگام است ولی به خاطر اجرای یک دستور خاص رخ می‌دهد (مثل page fault). پس تفاوت اصلی در «منشأ رویداد» و «همگام/ناهمگام بودن» و «عمدی بودن یا نبودن» است. همچنین Exception معمولاً به خطا/حفاظت/مدیریت حافظه مربوط است، ولی Interrupt غالباً سرویس‌دهی دستگاه‌هاست.

2. اصطلاح Reentrant Kernel به چه معناست؟ سناریویی را در xv6 شرح دهید که یک پروسه در حال اجرای یک Syscall است، ناگهان وقفه دیسک رخ می‌دهد و CPU مجبور می‌شود مجدداً وارد کد هسته شود. در این حالت اگر همگام‌سازی نباشد، چه خطری داده‌های کرنل را تهدید می‌کند؟

Reentrant Kernel یعنی کرنل می‌تواند همزمان چند مسیر اجرایی داشته باشد یا تودرتو دوباره وارد شود.

سناریو: پردازش داخل کرنل در حال اجرای یک syscall است و هنوز خارج نشده که وقفه دیسک رخ می‌دهد.

CPU اجرای syscall را قطع می‌کند و وارد هندلر وقفه دیسک می‌شود، یعنی دوباره در کد کرنل هستیم. اگر داده‌های مشترک کرنل قفل نداشته باشند، دو مسیر ممکن است همزمان یک ساختار را تغییر دهند. نتیجه می‌تواند Race Condition، خراب شدن لیست‌ها، بافرها، از دست رفتن به‌روزرسانی و کرش باشد. پس reentrancy یعنی کرنل باید برای ورود همزمان یا تودرتو آماده باشد و همگام‌سازی لازم دارد.

3. تفاوت Process Context و Interrupt Context را شرح دهید.

Process Context زمانی است که کرنل به نمایندگی از یک پردازش اجرا می‌شود مثل اجرای Syscall. در این حالت، پردازش مشخص است و می‌توان sleep کرد چون زمان‌بند می‌تواند پردازش دیگری را اجرا کند. Interrupt Context زمانی است که کد کرنل به خاطر وقفه اجرا می‌شود و به پردازش مشخصی وابسته نیست. در وقفه معمولاً باید سریع کار کرد و فقط کارهای ضروری انجام داد و بقیه را به بستر پردازش سپرد. در Interrupt Context بسیاری از عملیات مثل sleep خطرناک است چون ممکن است زمان‌بندی یا قفل‌ها به مشکل بخورد. پس تفاوت کلیدی وجود یا عدم وجود پردازش مالک، و مجاز بودن sleep و عملیات سنگین است.

4. چرا کدی که در Interrupt Context اجرا می‌شود (مثلاً هندلر تایمر)، به هیچ وجه نباید مسدود (Block) شود یا به خواب (Sleep) برود؟

هندلر وقفه باید سریع اجرا و تمام شود، چون تا پایان آن، رسیدگی به برخی وقفه‌ها یا ادامه اجرای عادی سیستم به تأخیر می‌افتد. اگر در Interrupt Context، sleep کنیم، عملاً CPU را در مسیر وقفه نگه می‌داریم و latency سیستم بالا می‌رود. از طرف دیگر، sleep معمولاً نیازمند زمان‌بند و تغییر زمینه است و در بستر وقفه انجام آن می‌تواند ناامن یا حتی غیرممکن باشد. خوابیدن هندلر می‌تواند باعث بن‌بست شود، چون ممکن است رویدادی که باید سیستم را جلو ببرد (مثل آزاد شدن قفل/بیدار شدن) خودش به رسیدگی وقفه‌ها یا ادامه اجرای مسیرهای دیگر نیاز داشته باشد. نتیجه ممکن است از دست رفتن رویدادها، کاهش شدید کارایی یا هنگ کردن سیستم باشد. بنابراین قاعده این است که هندلر فقط کار ضروری را انجام دهد و کارهای سنگین را به بستر پردازش منتقل کند.

```

418 sleep(void *chan, struct spinlock *lk)
419 {
420     struct proc *p = myproc();
421
422     if(p == 0)
423         panic("sleep");
424
425     if(lk == 0)
426         panic("sleep without lk");
427
428     // Must acquire ptable.lock in order to
429     // change p->state and then call sched.
430     // Once we hold ptable.lock, we can be
431     // guaranteed that we won't miss any wakeup
432     // (wakeup runs with ptable.lock locked),
433     // so it's okay to release lk.
434     if(lk != &ptable.lock){ //DOC: sleeplock0
435         acquire(&ptable.lock); //DOC: sleeplock1
436         release(lk);
437     }
438     // Go to sleep.
439     p->chan = chan;
440     p->state = SLEEPING;
441
442     sched();
443
444     // Tidy up.
445     p->chan = 0;
446
447     // Reacquire original lock.
448     if(lk != &ptable.lock){ //DOC: sleeplock2
449         release(&ptable.lock);
450         acquire(lk);
451     }
452 }

```

5. چرا برای محافظت از داده‌ها در Interrupt Context، معمولاً از ترکیب Spinlock و Disable Interrupts استفاده می‌شود؟

Spinlock برای نواحی بحرانی کوتاه مناسب است، چون سرشار sleep/wakeup و context switch ندارد و سریع رد می‌شود. اما مشکل اصلی در Interrupt Context این است که ممکن است همان CPU که قفل را گرفته، با وقفه دوباره وارد کرنل شود. اگر هندلر وقفه بخواند همان قفل را بگیرد، روی آن spin می‌کند و چون صاحب قفل همان CPU بوده و فعلاً قطع شده، قفل آزاد نمی‌شود. این حالت یک self-deadlock است و مخصوصاً در تک‌هسته سیستم را قفل می‌کند. Disable Interrupts روی همان CPU جلوی ورود تودرتو به هندلر در زمان نگه‌داشتن قفل را می‌گیرد. پس ترکیب Spinlock و غیرفعال‌سازی وقفه‌ها هم محافظت از داده را انجام می‌دهد و هم از بن‌بست ناشی از وقفه جلوگیری می‌کند.

6. سه رویکرد کلی برای همگام‌سازی وجود دارد. هر یک را در یک پاراگراف تعریف کرده و دو مزیت/معایب برای آن ذکر کنید:

- Spinlock (قفل چرخشی): مناسب برای انتظاراتهای کوتاه.
- Sleep lock (قفل خواب): مناسب برای انتظاراتهای طولانی (مثل I/O).
- Lock-free Programming: روشی پیشرفته بدون استفاده از قفل.

Spinlock یعنی اگر قفل آزاد نبود، درخواست‌کننده روی CPU منتظر می‌ماند و مدام وضعیت قفل را چک می‌کند تا آزاد شود. مزیتش این است که برای نواحی بحرانی کوتاه بسیار سریع است و هزینه context switch ندارد، اما عیبش این است که در انتظار طولانی CPU را هدر می‌دهد و در رقابت بالا کارایی را پایین می‌آورد. Sleeplock یعنی اگر قفل آزاد نبود، پردازنده می‌خوابد و CPU به کار دیگری می‌رسد و بعداً با wakeup برمی‌گردد. مزیتش مناسب بودن برای انتظاراتهای طولانی مثل I/O و جلوگیری از هدررفت CPU است، اما عیبش سربار sleep/wakeup و محدودیت استفاده در Interrupt Context است. Lock-free یعنی همگام‌سازی بدون قفل کلاسیک تا گلوگاه قفل کم شود. مزیتش کاهش deadlock های قفل‌محور و در برخی سناریوها مقیاس‌پذیری بهتر است، اما عیبش پیچیدگی بالا، دشواری اثبات صحت و امکان starvation و دیباگ سخت‌تر است.

7. در تابع acquire، قبل از گرفتن قفل، وقفه‌ها غیرفعال می‌شوند (cli)، چرا؟ فرض کنید وقفه‌ها باز باشند و یک پردازنده قفلی را بگیرد، سپس یک وقفه رخ دهد و هندلر وقفه بخواهد همان قفل را بگیرد. چه اتفاقی می‌افتد؟ (شرح سناریوی Deadlock در سیستم تک هسته ای).

اگر وقفه‌ها فعال باشند، ممکن است پردازنده روی CPU قفل را بگیرد و وارد ناحیه بحرانی شود. قبل از آزاد کردن قفل،

یک وقفه رخ می‌دهد و CPU وارد هندلر وقفه می‌شود و اجرای مسیر اصلی قطع می‌گردد. حالا اگر هندلر برای

دسترسی به همان داده مشترک بخواهد همان قفل را acquire کند، چون قفل آزاد نیست شروع به spin می‌کند.

در سیستم تک‌هسته، کدی که باید قفل را آزاد کند همان کد اصلی است که فعلاً اجرا نمی‌شود، پس قفل هیچ‌وقت

آزاد نمی‌شود. نتیجه یک self-deadlock قطعی است و سیستم عملاً گیر می‌کند.

Cli قبل از acquire باعث می‌شود هنگام نگه‌داشتن قفل، همان CPU نتواند با وقفه وارد مسیر تودرتویی شود که

دوباره همان قفل را می‌خواهد.

```

24 void
25 acquire(struct spinlock *lk)
26 {
27     pushcli(); // disable interrupts to avoid deadlock.
28     if(holding(lk))
29         panic("acquire");
30
31     // The xchg is atomic.
32     while(xchg(&lk->locked, 1) != 0)
33         ;
34
35     // Tell the C compiler and the processor to not move loads or stores
36     // past this point, to ensure that the critical section's memory
37     // references happen after the lock is acquired.
38     __sync_synchronize();
39
40     // Record info about lock acquisition for debugging.
41     lk->cpu = mycpu();
42     getcallerpcs(&lk, lk->pcs);
43 }
44
45 // Release the lock.
46 void
47 release(struct spinlock *lk)
48 {
49     if(!holding(lk))
50         panic("release");
51
52     lk->pcs[0] = 0;
53     lk->cpu = 0;
54
55     // Tell the C compiler and the processor to not move loads or stores
56     // past this point, to ensure that all the stores in the critical
57     // section are visible to other cores before the lock is released.
58     // Both the C compiler and the hardware may re-order loads and
59     // stores; __sync_synchronize() tells them both not to.
60     __sync_synchronize();
61
62     // Release the lock, equivalent to lk->locked = 0.
63     // This code can't use a C assignment, since it might
64     // not be atomic. A real OS would use C atomics here.
65     asm volatile("movl $0, %0" : "+m" (lk->locked) : );
66
67     popcli();
68 }

```

8. چرا xv6 به جای استفاده مستقیم از cli و sti، از توابع pushcli و popcli استفاده می کند؟ این توابع چگونه

مشکل "تودرتو بودن نواحی بحرانی" (Nested Critical Sections) را حل می کنند؟

اگر از cli/sti مستقیم استفاده شود، در نواحی بحرانی تودرتو ممکن است sti زودتر از موعد اجرا شود و وقفه‌ها در حالی روشن شوند که هنوز ناحیه بحرانی بیرونی تمام نشده است. مثلاً ناحیه بیرونی cli کرده، سپس ناحیه داخلی هم cli می کند، اما اگر داخلی sti کند، وقفه‌ها روشن می شوند و داده‌ها هنوز باید محافظت شوند.

Pushcli معمولاً علاوه بر خاموش کردن وقفه، یک شمارنده تودرتویی (nesting depth) را افزایش می دهد و وضعیت قبلی وقفه را هم نگه می دارد. popcli شمارنده را کم می کند و فقط وقتی شمارنده به صفر رسید اجازه می دهد وقفه‌ها به وضعیت قبلی برگردند. بنابراین روشن شدن وقفه‌ها دقیقاً زمانی رخ می دهد که آخرین ناحیه بحرانی تمام شده باشد. این کار هم از race جلوگیری می کند و هم رفتار صحیح در بخش‌های تودرتو را تضمین می کند.

1. چرا زمانی که یک پردازنده یک spinlock را در اختیار دارد، حتماً باید وقفه‌ها روی آن پردازنده غیرفعال باشند؟

وقتی CPU یک spinlock را گرفته، نباید همان CPU با وقفه وارد مسیری شود که دوباره همان قفل را بخواهد. اگر وقفه فعال باشد، هندلر وقفه ممکن است برای همان ساختار داده نیاز به همان قفل داشته باشد. هندلر در این حالت روی قفل spin می‌کند، اما صاحب قفل همان CPU است و مسیر اصلی که باید قفل را آزاد کند قطع شده است. در تک‌هسته این دقیقاً یک self-deadlock قطعی است و سیستم متوقف می‌شود. حتی در چندهسته هم می‌تواند باعث گیر کردن هسته مربوط و اختلال جدی شود. پس غیرفعال کردن وقفه‌ها روی همان CPU یک الزام عملی برای جلوگیری از بن‌بست است.

2. اگر وقفه فعال بماند و هندلر وقفه (Interrupt Handler) بخواهد همان قفل را بگیرد، چه نوع بن‌بستی (Deadlock) رخ می‌دهد؟ (با رسم شکل یا مثال توضیح دهید)

این یک بن‌بست تودرتو (self-deadlock) است که روی همان CPU اتفاق می‌افتد. مالک قفل کد اصلی است که قبل از آزاد کردن قفل با وقفه قطع می‌شود. هندلر وقفه به خاطر نیاز به همان داده مشترک تلاش می‌کند همان قفل را acquire کند و چون قفل آزاد نیست در حلقه spin می‌ماند. از آنجا که CPU تک هسته‌ای است، هیچ مسیر دیگری برای آزادسازی قفل وجود ندارد و پیشروی متوقف می‌شود. در عمل سیستم یا همان هسته قفل می‌شود و کارایی و پاسخ‌دهی به شدت افت می‌کند. بنابراین این deadlock نتیجه “قطع شدن مالک قفل و تلاش مجدد همان CPU برای گرفتن همان قفل” است.

مثال: فرض کنید در کرنل، یک spinlock به نام L برای محافظت از یک صف آماده یا یک شمارنده استفاده می‌شود. پردازنده P وارد کرنل می‌شود و در مسیر اجرای یک syscall یا کد زمان‌بند، قفل L را acquire می‌کند

و هنوز آن را آزاد نکرده است. در همین لحظه وقفه تایمر رخ می دهد و چون وقفه ها فعال اند، CPU اجرای P را قطع می کند و وارد handler تایمر می شود. داخل handler تایمر برای به روز رسانی همان ساختار (مثلاً همان صف یا شمارنده) دوباره نیاز به قفل L است، پس handler تلاش می کند L را acquire کند. اما L قبلاً توسط همان CPU در کد P گرفته شده و آزاد نشده است؛ بنابراین handler وارد حلقه spin می شود و منتظر آزاد شدن L می ماند. مشکل اینجاست که تنها کسی که می تواند L را آزاد کند همان کد قبلی (P) است، ولی آن کد دیگر اجرا نمی شود چون CPU در handler گیر کرده است. پس سیستم در یک حلقه بی پایان گیر می کند و قفل هیچ وقت آزاد نمی شود.

3. با توجه به اسلایدهای درس، اگر چندین هسته پردازنده بخواهند به طور مداوم یک متغیر مشترک سراسری (مثلاً Globalcounter) را تغییر دهند، پدیده ای رخ می دهد که باعث کندی سیستم می شود. پروتکل های همگام سازی حافظه نهان (مانند MESI) در این شرایط چه می کنند؟

وقتی چند CPU دائماً روی یک متغیر مشترک می نویسند، کش لاین مربوط به آن متغیر بین هسته ها مدام جابه جا می شود که به آن cache line bouncing یا ping-pong می گویند. این پدیده باعث می شود زمان زیادی صرف هماهنگ سازی کش ها به جای انجام کار مفید شود. در پروتکل های coherence مثل MESI، برای نوشتن لازم است کش لاین در حالت انحصاری در اختیار یک CPU باشد. بنابراین قبل از نوشتن، کش های دیگر باید آن خط را invalidate کنند تا تضاد نوشتن رخ ندهد. این invalidate شدن های مکرر ترافیک سنگین coherence ایجاد می کند و latency را بالا می برد. نتیجه نهایی کند شدن سیستم و کاهش مقیاس پذیری در بارهای رقابتی است.

4. چرا استفاده از متغیرهای محلی (Per-CPU) این مشکل را تا حد زیادی کاهش می دهد؟ به طور خلاصه توضیح دهید چگونه نگه داشتن شمارنده ها به صورت per-CPU می تواند تعداد invalidation های کش را کم کند.

در روش CPU-per به جای یک متغیر مشترک، برای هر CPU یک نسخه جدا از متغیر نگه می داریم. در این حالت هر CPU فقط روی داده خودش می نویسد و معمولاً همان داده در کش همان CPU باقی می ماند. چون CPU های دیگر روی همان کش لاین نمی نویسند، invalidate شدن های مکرر و bouncing تقریباً حذف می شود. این کار ترافیک coherence را کم می کند و کارایی در سیستم های چند هسته ای بهتر می شود. بعداً در یک نقطه امن می توان مقادیر per-CPU را جمع زد و مقدار کلی را ساخت. بنابراین CPU-per با محلی سازی نوشتن ها، هزینه هماهنگ سازی کش را شدیداً کاهش می دهد.

5. تفاوت اصلی بین spinlock (دسته اول توابع در xv6) و sleeplock (دسته دوم توابع) در چیست؟ کدام یک باعث «انتظار مشغول» (Busy Waiting) می شود و کدام یک پردازنده را به پروسس دیگری واگذار می کند؟

در spinlock اگر قفل آزاد نباشد، CPU منتظر می ماند و مدام چک می کند؛ این همان busy waiting است و CPU مصرف می شود. این روش برای نواحی بحرانی کوتاه مفید است چون context switch ندارد و سریع تمام می شود. در sleeplock اگر قفل آزاد نباشد، پردازنده به خواب می رود و CPU به پردازنده دیگری داده می شود، پس هدررفت CPU کمتر است sleeplock. برای انتظارهای طولانی مثل I/O مناسب تر است، چون busy waiting در این حالت بسیار پرهزینه می شود. عیب sleeplock این است که سربار sleep/wakeup و context switch دارد و در Interrupt Context مناسب نیست. در نتیجه busy waiting مربوط به spinlock است و واگذاری CPU مربوط به sleeplock


```

24 void
25 acquire(struct spinlock *lk)
26 {
27     pushcli(); // disable interrupts to avoid deadlock.
28     if(holding(lk))
29         panic("acquire");
30
31     // The xchg is atomic.
32     while(xchg(&lk->locked, 1) != 0)
33         ;
34
35     // Tell the C compiler and the processor to not move loads or stores
36     // past this point, to ensure that the critical section's memory
37     // references happen after the lock is acquired.
38     __sync_synchronize();
39
40     // Record info about lock acquisition for debugging.
41     lk->cpu = mycpu();
42     getcallerpcs(&lk, lk->pcs);
43 }

```

بخش سوم: قفل های اولویت دار

1. آیا در طراحی plock (قفلی که در ادامه پیاده سازی می کنید)، امکان گرسنگی برای پردازش های با اولویت پایین وجود دارد؟ یک سناریوی دقیق (Scenario) بنویسید که در آن یک پردازش با اولویت پایین (Priority Low) درخواست قفل می کند، اما با وجود اینکه قفل بارها توسط دیگران گرفته و آزاد می شود، این پردازش هرگز موفق به دریافت قفل نمی شود.

بله، چون plock قفل را همیشه به بالاترین اولویت می دهد، پردازش های کم اولویت ممکن است برای مدت نامحدود منتظر بمانند. سناریو: پردازش L با اولویت پایین درخواست قفل می دهد و وارد صف انتظار می شود. قفل هر بار آزاد می شود اما قبل از اینکه L انتخاب شود، پردازش های با اولویت بالاتر H1 و H2 و ... وارد صف می شوند یا از قبل در صف حضور دارند. هنگام release، plock همیشه یکی از پردازش های H را انتخاب می کند، چون اولویتشان بالاتر است. بنابراین L با وجود اینکه مدت ها منتظر است، باز هم انتخاب نمی شود و عقب می افتد. اگر این ورود پردازش های با اولویت بالا ادامه پیدا کند، L ممکن است هرگز قفل را نگیرد؛ این دقیقاً starvation است.

2. یک مکانیزم دیگر برای مدیریت صف، استفاده از قفل بلیطی (Ticket Lock) است که شبیه سیستم نوبت‌دهی نانوایی عمل می‌کند (FIFO). نحوه کارکرد Ticket Lock را به صورت مفهومی توضیح دهید.

Ticket lock شبیه سیستم نوبت‌دهی FIFO است و هدفش این است که ترتیب گرفتن قفل کاملاً منصفانه و قابل پیش‌بینی باشد. هر درخواست‌کننده یک شماره بلیط می‌گیرد با یک عمل مثل fetch-and-increment روی next_ticket.

سپس تا زمانی که now_serving برابر شماره بلیط خودش نشود منتظر می‌ماند. وقتی دارنده قفل کارش تمام شد، با افزایش now_serving نوبت را به نفر بعدی می‌دهد. در نتیجه هیچ‌کس نمی‌تواند از صف جلو بزند و هرکس دقیقاً به ترتیب ورود سرویس می‌گیرد. این روش جلوی گرسنگی ناشی از شانس زمان‌بندی را تا حد زیادی می‌گیرد. همچنین رفتار قفل قابل تحلیل‌تر و منظم‌تر می‌شود چون FIFO تضمین است.

3. قفل اولویت‌دار (plock) را با قفل بلیطی (ticket lock) از نظر انصاف (کدام یک عادلانه‌تر رفتار می‌کند؟) پیچیدگی (پیاده‌سازی کدام یک سربار محاسباتی کمتری دارد؟) احتمال گرسنگی: کدام یک در برابر گرسنگی ایمن است؟

از نظر Fairness، ticket lock معمولاً عادلانه‌تر است چون FIFO واقعی دارد و هرکس به نوبت می‌رسد، اما plock عمداً اولویت را ترجیح می‌دهد و ممکن است یک پردازش دیرتر آمده زودتر قفل بگیرد.

از نظر Complexity، ticket lock اغلب ساده‌تر است چون چند متغیر شمارنده و عملیات اتمیک دارد، ولی plock نیازمند صف انتظار، گره‌ها، و انتخاب بیشینه اولویت است که سربار و پیچیدگی بیشتری دارد.

از نظر Starvation، ticket lock معمولاً در برابر گرسنگی مقاوم‌تر است چون FIFO است و نوبت تضمین می‌شود. اما plock ذاتاً در برابر starvation آسیب‌پذیر است، چون ورود مداوم پردازش‌های با اولویت بالا می‌تواند پردازش‌های کم‌اولویت را نامحدود عقب بیندازد. مگر اینکه مکانیزم‌های مکمل مثل aging یا محدودیت خدمت‌دهی اضافه شود، این خطر پابرجاست.